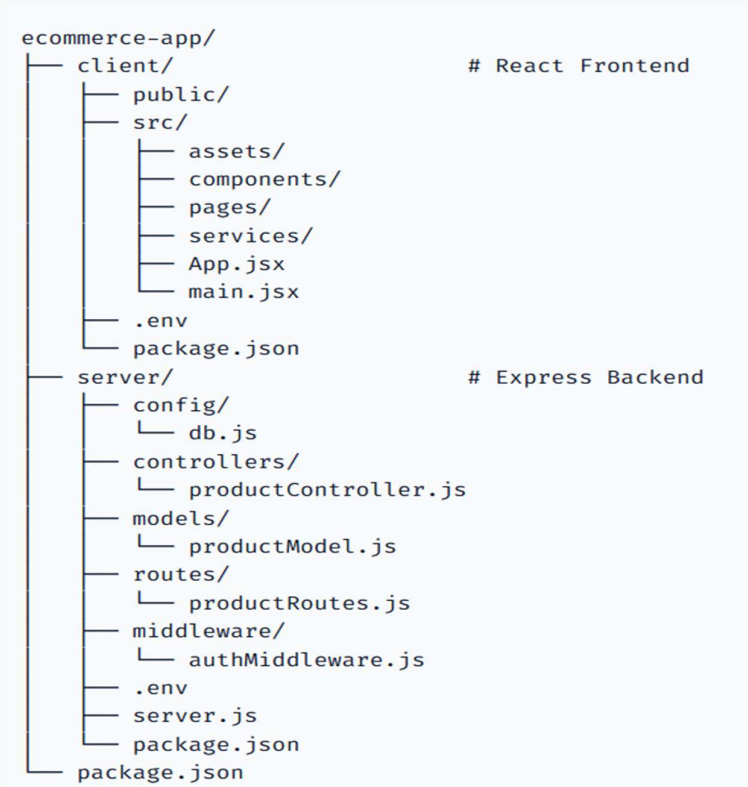
Unit I Assignment
Full Stack Web Development
// Khushi Choudhary
// CU240251564

1: Full Stack Folder Structure & Layer Justification



Layer Separation Justification:
Direct coupling between routes/ and models/ bypasses the controller layer, violating the Separation of Concerns (MVC pattern).
Controllers must mediate between routes and models to handle business logic, request validation, and response formatting cleanly.

2: Hardcoded Production URL Fix
Why Hardcoded Localhost Fails in Production:
localhost points strictly to the client machine running the browser, causing deployed users to request port 5000 on their own devices.
In production, backend APIs run on external hostnames or remote IP addresses, rendering hardcoded local URLs unreachable.

Rewritten Fetch Line using Environment Variable
// Create React App Standard
fetch(`${process.env.REACT_APP_API_URL}/api/products`)
// Vite + React Standard
fetch(`${import.meta.env.VITE_API_URL}/api/products`)

3: Backend Environment Configuration (Backend .env File)
PORT=5000

```
DB_URL=mysql://root:password@localhost:3306/ecommerce_db
JWT_SECRET=super_secret_jwt_key_12345
```

Dotenv Loading Code (server.js)

```
require('dotenv').config();
const PORT = process.env.PORT || 5000;
const DB_URL = process.env.DB_URL;
const JWT_SECRET = process.env.JWT_SECRET;
```

Why .env should NEVER be committed to Git:

The .env file contains sensitive secrets and database credentials that would be publicly exposed if committed to version control.

4: Express CORS Error Fix (Express Middleware Code)

```
const express = require('express');
const cors = require('cors');
const app = express();
// Enable CORS middleware for all cross-origin requests
app.use(cors());
```

Why Browser Blocks Request:

Browsers enforce the Same-Origin Policy (SOP), blocking client-side HTTP requests made to a different origin (domain, protocol, or port) unless explicit CORS headers are returned by the server.

5: Book Library REST API Contract

Endpoint	Method	Request Body	Response
/api/books	POST	{"title": "1984", "author": "George Orwell", "year": 1949}	**201 Created** {"id": 1, "title": "1984", "author": "George Orwell", "year": 1949}
/api/books	GET	*None*	**200 OK** [{"id": 1, "title": "1984", "author": "George Orwell", "year": 1949}]
/api/books/:id	PUT	{"title": "1984", "author": "George Orwell", "year": 1950}	**200 OK** {"id": 1, "title": "1984", "author": "George Orwell", "year": 1950}
/api/books/:id	DELETE	*None*	**200 OK** {"message": "Book deleted successfully"}

6: Database Connection Reuse & Pooling

Rewritten Code (Connection Created Once Outside Route Handlers)

```
const express = require('express');
const mysql = require('mysql2');
const app = express();
// Connection pool created ONCE outside route handlers
const db = mysql.createPool({
```

```
  host: 'localhost',
  user: 'root',
  database: 'ecommerce_db'
});
app.get('/products', (req, res) => {
  db.query('SELECT * FROM products', (err, rows) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json(rows);
  });
});
```

Why original code is a performance problem:
Opening a fresh database connection on every request creates heavy CPU and network overhead due to repetitive TCP/handshake setup.
Under high traffic, creating connections repeatedly exhausts connection limits and quickly crashes the database server.

7: Concurrently Full Stack Scripts
package.json Scripts Section

```
"scripts": {
  "server": "nodemon server.js",
  "client": "npm start --prefix client",
  "dev": "concurrently \"npm run server\" \"npm run client\""
}
```

8: Frontend API Key Security Fix (Rewritten Code using Environment Variable)

```
const API_KEY = process.env.REACT_APP_WEATHER_API_KEY; // Or
import.meta.env.VITE_WEATHER_API_KEY
const response = await fetch(
  `https://api.weather.com/data?key=${API_KEY}`
);
```

Security Risk Explanation:
Client-side React code is compiled into bundle files delivered directly to the user, making hardcoded API keys readable by anyone via browser DevTools.
Exposed keys allow unauthorized users to steal quota, access restricted resources, or incur unexpected financial costs on your account.

9: Full Request-Response Cycle Pseudocode

```
// STEP 1: UI Event (Frontend)
// User fills out form and clicks 'Login' button -> onSubmit handler triggers
event.preventDefault().

// STEP 2: HTTP Request Dispatch (Frontend)
// Frontend extracts username & password and sends asynchronous POST fetch/axios request
to '/api/login'.
```

// STEP 3: Server Receipt & Middleware (Backend)
// Express server catches request -> express.json() parses raw JSON payload into req.body object.

// STEP 4: Controller Logic & Validation (Backend)
// Request hits auth controller -> verifies required fields exist in req.body.

// STEP 5: Database Query Execution (Backend -> MySQL)
// Controller calls User model -> executes SQL query: `SELECT * FROM users WHERE username = ?`.

// STEP 6: DB Response & Password Hash Check (Backend)
// MySQL returns user record -> backend compares submitted password hash using bcrypt.compare().

// STEP 7: Token Generation & HTTP Response (Backend -> Frontend)
// Server generates JWT authentication token and responds with HTTP status 200 OK and JSON body { token, user }.

// STEP 8: UI Update & State Persistence (Frontend)
// Frontend receives 200 OK -> stores JWT token in localStorage/cookies, updates auth state, and redirects to dashboard.

10: Express POST Body Undefined Error Fix

Missing Line Identification:
The server is missing the built-in JSON body parsing middleware (app.use(express.json())). Without it, Express does not parse incoming JSON payloads, resulting in req.body being undefined.

Corrected Code:
```
const express = require('express');
const app = express();

// FIX: Enable JSON body parsing middleware before routes
app.use(express.json());

app.post('/login', (req, res) => {
  const { username, password } = req.body;
  res.send(`Welcome ${username}`);
});

app.listen(3000);
```